

Кафедра «Информационные системы»

ОТЧЕТ

По СРОN№1

Тема: Food Nutrition Calculator Продукты, порции, калории и БЖУ; расчёт пищевой ценности.

По дисциплине : Визуальное программирование

Вариант : №19

Выполнил(а):

Симониди Анастас

Группа: ИС 24-22

Проверил(а):

Рашидинов Д.Р

[Алматы], 2026

1. Задача и цель проекта

Цель: Разработка десктопного приложения на JavaFX для учёта продуктов питания, расчёта их пищевой ценности (КБЖУ) и ведения дневника питания.

Задача: Создать приложение с графическим интерфейсом, реализующее функции добавления, редактирования, удаления и поиска продуктов, а также подсчёт итоговых калорий и макронутриентов на основе веса порции.
Требования к реализации:

Использование FXML для разметки и Scene Builder для проектирования.

Разделение логики (MVC-паттерн).

Реализация CRUD-операций (Create, Read, Update, Delete).

Наличие валидации ввода, диалоговых окон и горячих клавиш.

2. Структура проекта и архитектура

Проект построен на основе паттерна MVC (Model-View-Controller) и сборщика Maven.

Model (Модель): Класс FoodItem.java. Содержит данные о продукте (название, категория, калорийность, белки, жиры, углеводы, вес) и использует свойства JavaFX (StringProperty, DoubleProperty) для автоматической привязки к таблице.

View (Представление): FXML-файлы (main-view.fxml, food-form.fxml), созданные в Scene Builder. Отвечают за визуальную часть.

Controller (Контроллер): Классы FoodController.java (управление главным окном) и FoodFormController.java (управление окном добавления/редактирования).

Точка входа: Launcher.java и MainApp.java.

Структура пакетов: Все классы находятся в пакете com.example.srovariant19.

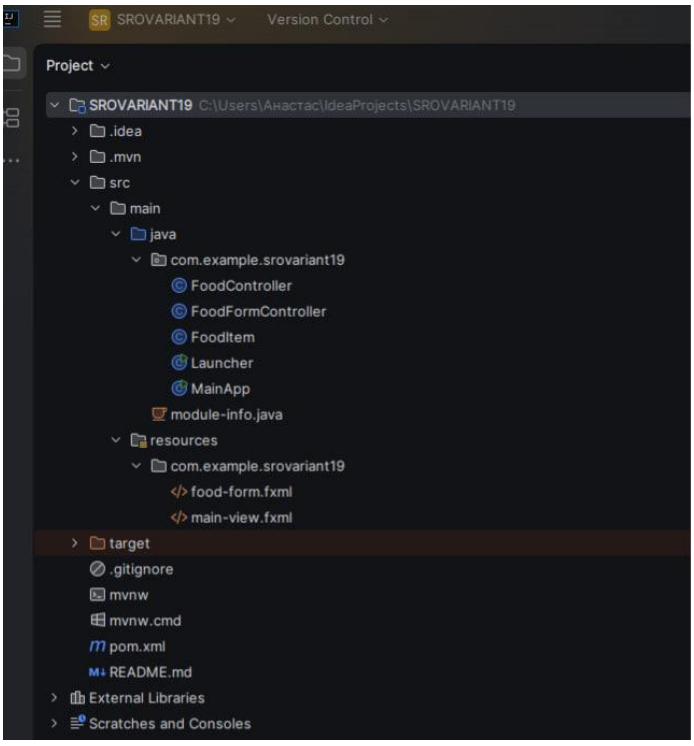


Рис 1.1 Стркута проекта

3. Макеты и описание интерфейса

(Вставьте скриншот главного окна)
Главное окно разделено с помощью BorderPane:

Сверху: MenuBar (Файл, Правка, Вид, Помощь) и ToolBar с кнопками быстрого доступа.

В центре: TableView, отображающая список продуктов.

Снизу: строка состояния (отображает количество записей и итоговые значения КБЖУ).
(Вставьте скриншот окна формы)
Окно добавления/редактирования реализовано через модальное окно Stage и использует GridPane для аккуратного размещения полей ввода (TextField, ComboBox, Spinner).

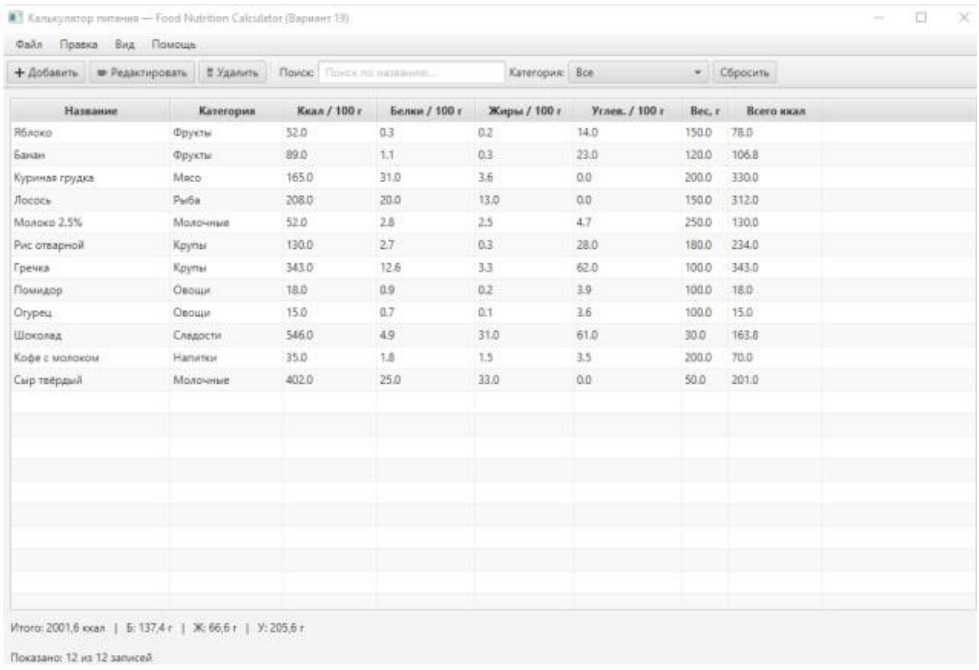


Рис 1.2 Интерфейс проекта

4. Описание компонентов и событий

TableView: Основной компонент, привязанный к ObservableList. Колонки привязаны к свойствам модели (например, nameCol.setCellValueFactory(c -> c.getValue().nameProperty())).

Фильтрация: Используется FilteredList. Слушатели (Listeners) на TextField (поиск) и ComboBox (категория) динамически обновляют список.

События (Event Handling): Обработка нажатий кнопок реализована через аннотацию @FXML. Например, метод onAdd() открывает новое окно, а onDelete() вызывает диалог подтверждения Alert.

Горячие клавиши: В MenuBar заданы акселераторы (Ctrl+N, Delete, Ctrl+Q), а также добавлена комбинация Ctrl+F для фокуса на строке поиска.

5. Примеры кода

1. Валидация ввода (из FoodFormController.java):

```
130         private double parseNonNegative(String raw, String label) {  4 usages
131             if (raw == null || raw.isBlank()) {
132                 throw new IllegalArgumentException(label + ": поле обязательно для заполнения.");
133             }
134             try {
135                 double v = Double.parseDouble(raw.replace( oldChar: ',', newChar: '.').trim());
136                 if (v < 0) throw new NumberFormatException();
137                 if (v > 10000) throw new IllegalArgumentException(label + ": значение слишком большое.");
138                 return v;
139             } catch (NumberFormatException e) {
140                 throw new IllegalArgumentException(label + ": введите неотрицательное число.");
141             }
142         }
143     }
```

Рис 1.3

2. Подсчёт итогов (из FoodController.java):

```
private void updateStatus() {  6 usages
    int shown = filteredItems == null ? 0 : filteredItems.size();
    double cal = 0, p = 0, f = 0, c = 0;
    if (filteredItems != null) {
        for (FoodItem it : filteredItems) {
            cal += it.getTotalCalories();
            p   += it.getTotalProteins();
            f   += it.getTotalFats();
            c   += it.getTotalCarbs();
        }
    }
```

Рис 1.4

6. Тестирование и обработка ошибок

Были проведены следующие виды тестирования:

Добавление записи: Ввод корректных данных приводит к появлению записи в таблице.

Валидация (негативный тест): Попытка ввести текст в поле «Калорийность» или оставить пустым поле «Название» вызывает Alert с ошибкой. Приложение не вылетает (обработка через try-catch).

Поиск и фильтрация: Ввод буквы «Я» в поиск оставляет только яблоко, при этом итоговые калории пересчитываются.

Удаление: При нажатии «Удалить» появляется диалог подтверждения. Отмена действия оставляет запись на месте.

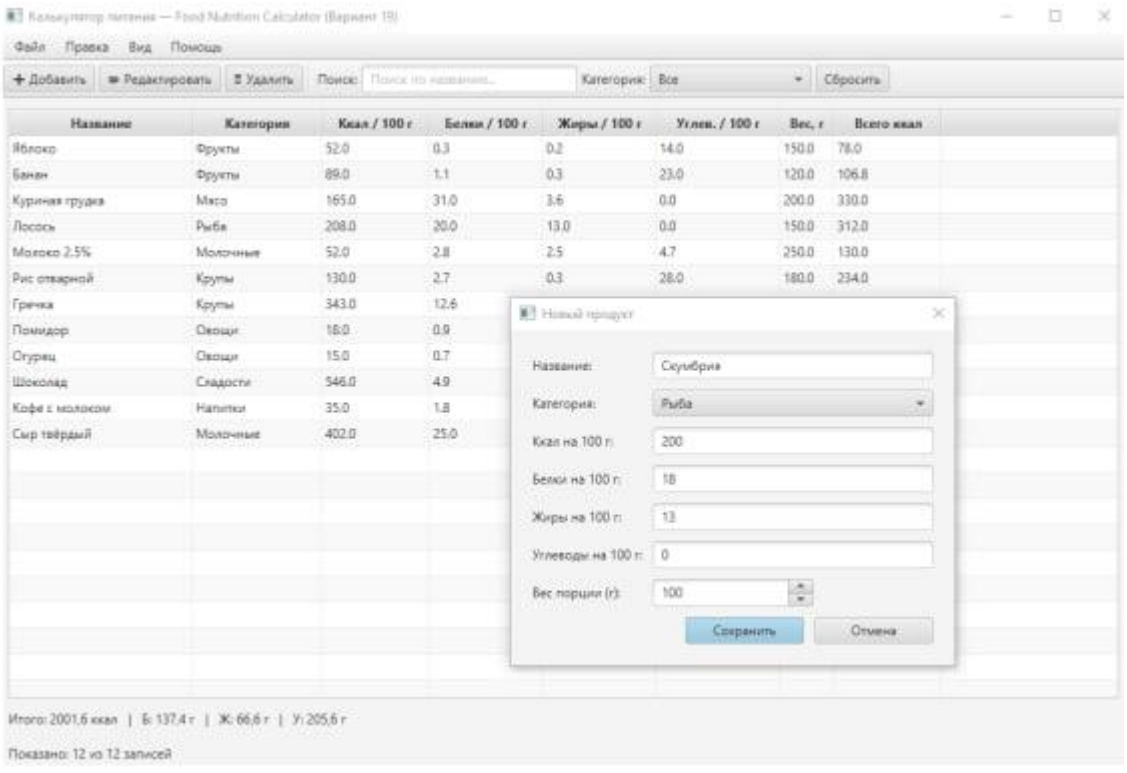


Рис 1.5 Добавление продукта

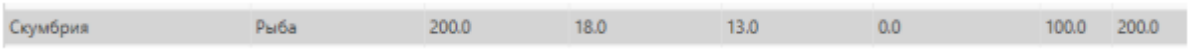


Рис 1.7 Продукт успешно отображается

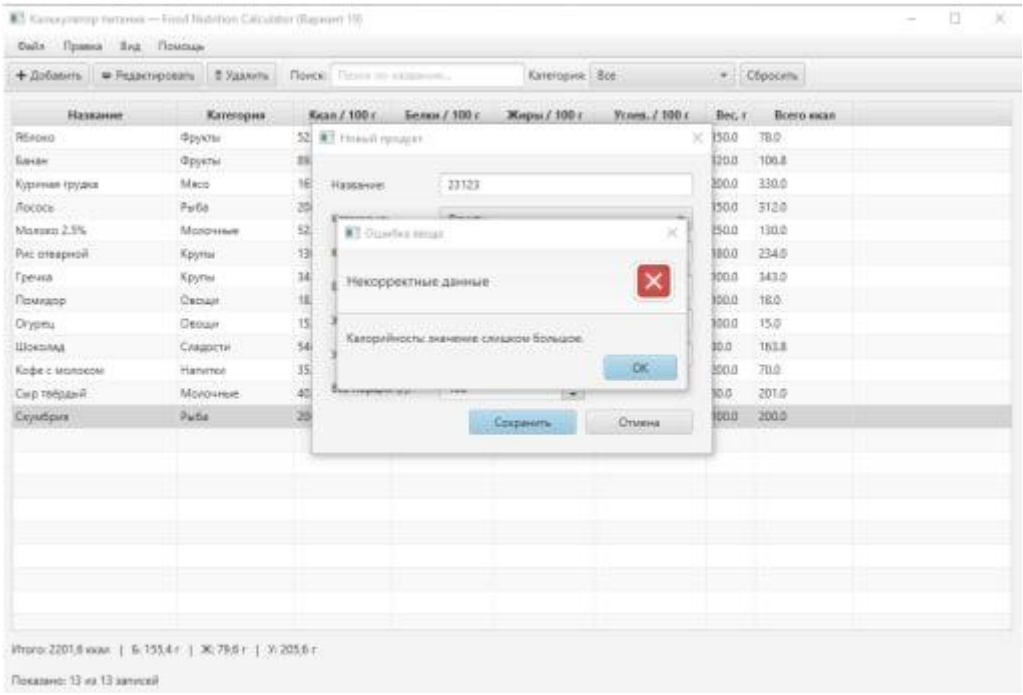


Рис 1.8 ошибка добавления продукта при вводе некорректных данных

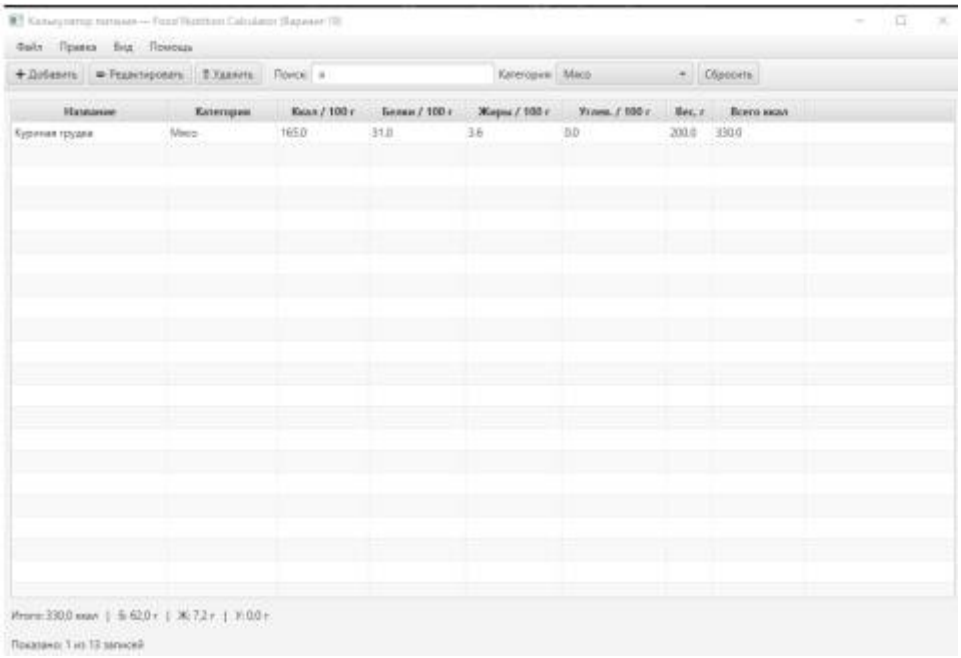


Рис 1.9 Фильтрация данных

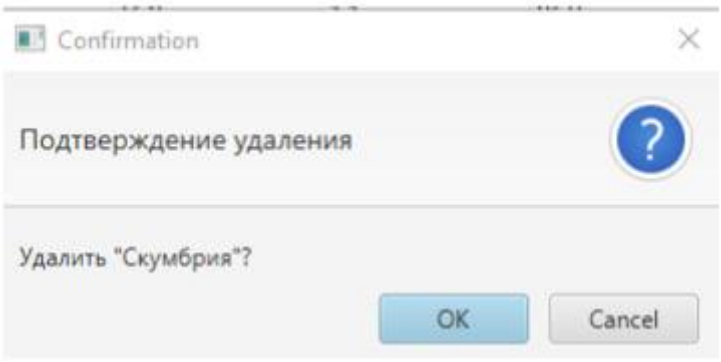


Рис 1.10 Подтверждение удаления продукта

7. Вывод

В ходе выполнения мини-проекта были закреплены навыки работы с JavaFX, Scene Builder, FXML и паттерном MVC. Освоено связывание данных (Binding) через свойства JavaFX, работа с коллекциями FilteredList, настройка обработчиков событий, горячих клавиш и валидация пользовательского ввода. Приложение полностью соответствует техническому заданию.